EX07 - Taylor de e^x: estado, factorial y expresion simbolica

Por que el bucle empieza en n = 0, como se mantiene n! sin recalcularlo y como mostrar la serie en el orden matematico que construimos.

---

1. La idea matematica

Queremos construir la aproximacion de Maclaurin de la exponencial. Si truncamos en orden N, la expresion es:

$$e^x \approx \sum_{n=0}^{N} x^n / n!$$

El punto importante es que el codigo debe reproducir esa estructura: para cada n, calcular un termino x^n/n! y sumarlo al acumulador de la serie.

```
ex = sp.Integer(0)
fact = 1
terms = []

for n in range(0, N + 1):

    term = x**n / fact

    ex += term
    terms.append(term)

    expression_to_show = " + ".join(str(term) for term in terms)

    print(f"n = {n} | ex = {expression_to_show}")

    fact *= n + 1
```

> Lectura en una frase
>
> En cada vuelta usamos el factorial que corresponde al termino actual, sumamos ese termino a la serie y, al final, preparamos el factorial que necesitara la siguiente vuelta.

2. Que representa cada variable

Variable	Inicial	Papel	Interpretacion
ex	sp.Integer(0)	Acumulador simbolico de la serie	Va guardando 1, 1+x, 1+x+x^2/2, ...
fact	1	Factorial requerido por el termino actual	Al inicio de la iteracion n debe valer n!
terms	[]	Lista solo para visualizacion	Guarda cada termino en el orden en que lo generamos
n	0..N	Contador / orden del termino	Selecciona x^n/n!
term	x**n/fact	Termino actual	La contribucion que se suma en esa iteracion

Por que `ex = sp.Integer(0)`

El valor matematico sigue siendo cero, pero ahora es un objeto de SymPy. Eso garantiza que el acumulador pertenece desde el principio al mundo simbolico y dispone de operaciones como `subs()`. Con este bucle que siempre incluye n=0 no seria estrictamente imprescindible, porque al sumar el primer termino SymPy convertiria la expresion; aun asi, es una inicializacion mas coherente y robusta.

> Valor matematico vs tipo computacional
>
> `0` y `sp.Integer(0)` representan el mismo numero, pero no son el mismo tipo de objeto. El segundo conoce operaciones simbolicas de SymPy.

3. La clave: fact siempre representa n!

Esta es la parte mas importante del algoritmo. Al comienzo de cada iteracion queremos que se cumpla la regla:

fact = n! (al entrar en la iteracion n)

Esta propiedad se llama invariante del bucle: una relacion que sigue siendo verdadera cada vez que empieza una nueva vuelta. Gracias a ella no necesitamos recalcular n! desde cero.

n	fact al entrar	term	fact tras actualizar
0	1 = 0!	x^0 / 1 = 1	1 * (0+1) = 1 = 1!
1	1 = 1!	x / 1 = x	1 * (1+1) = 2 = 2!
2	2 = 2!	x^2 / 2	2 * (2+1) = 6 = 3!
3	6 = 3!	x^3 / 6	6 * (3+1) = 24 = 4!
4	24 = 4!	x^4 / 24	24 * (4+1) = 120 = 5!

> Por que la actualizacion va al final
>
> Si hicieramos `fact *= n + 1` antes de calcular `term`, en la iteracion n=1 ya estariamos usando 2! en vez de 1!. El orden de las instrucciones forma parte del algoritmo.

4. Por que n = 0 cabe dentro del mismo bucle

La serie empieza realmente en n=0. Ese primer termino no necesita una excepcion:

$$x^0 / 0! = 1 / 1 = 1$$

Si inicializamos `fact = 1` y empezamos con `range(0, N + 1)`, la primera iteracion produce exactamente ese termino. Por eso podemos eliminar un `if N > 0` y mantener toda la serie dentro de un unico flujo.

> Principio de simplificacion
>
> Si el algoritmo general ya resuelve correctamente un caso particular, no hace falta crear una rama especial para ese caso.

Comparacion con la version anterior

Antes

```
ex = 1
fact = 1

for n in range(1, N + 1):
    fact *= n
    ex += x**n / fact

if N > 0:
    ex = ex.subs(x, xp)
```

Ahora

```
ex = sp.Integer(0)
fact = 1

for n in range(0, N + 1):
    term = x**n / fact
    ex += term
    fact *= n + 1

ex_numeric = ex.subs(x, xp)
```

5. terms y join(): controlar solo la visualizacion

SymPy reorganiza las sumas en un orden algebraico canonico. Por eso puede mostrar:

```
x**3/6 + x**2/2 + x + 1
```

aunque nosotros hayamos generado los terminos como:

```
1 + x + x**2/2 + x**3/6
```

Las dos expresiones son matematicamente identicas. Si queremos ver el orden pedagogico en que se construye la serie, usamos una lista separada:

```
terms.append(term)
expression_to_show = " + ".join(str(term) for term in terms)
```

`terms.append(term)` guarda el objeto simbolico actual al final de la lista.

`str(term) for term in terms` recorre la lista y convierte cada termino en texto.

`" + ".join(...)` pega esos textos insertando el separador `+` entre ellos.

> Importante
>
> `terms` y `expression_to_show` no participan en el calculo matematico. Son solo una capa de presentacion para que nosotros podamos leer la serie en el orden deseado.

6. Ejecucion completa para N = 4

n	fact usado	term	ex tras sumar	fact preparado
0	1	1	1	1!
1	1	x	1 + x	2!
2	2	x^2/2	1 + x + x^2/2	3!
3	6	x^3/6	1 + x + x^2/2 + x^3/6	4!
4	24	x^4/24	1 + x + x^2/2 + x^3/6 + x^4/24	5!

La secuencia conceptual en cada vuelta es:

1) usar el estado actual -> 2) calcular el termino -> 3) acumularlo -> 4) mostrarlo -> 5) preparar el estado de la siguiente iteracion

7. Patron recomendado para tu EX07

```
from math import exp
import sympy as sp

N = 5
x = sp.symbols("x")
xp = 0.5

ex_symbolic = sp.Integer(0)
fact = 1
terms = []

for n in range(0, N + 1):
    term = x**n / fact
    ex_symbolic += term
    terms.append(term)

    expression_to_show = " + ".join(str(t) for t in terms)
    print(f"n = {n} | ex = {expression_to_show}")

    fact *= n + 1

ex_numeric = ex_symbolic.subs(x, xp)
analytical_result = exp(xp)
```

Separar `ex_symbolic` de `ex_numeric` conserva simultaneamente la formula y su evaluacion. Esto sera especialmente util cuando empecemos a representar curvas, comparar errores y convertir el codigo en funciones reutilizables.

> Learning tracker
>
> Conceptos nuevos: objeto simbolico de SymPy, invariante de bucle, estado actual vs estado futuro, orden de actualizacion, representacion separada del calculo, listas como historial, generator expression y `join()`.

8. Errores tipicos a evitar

Error	Por que importa
Actualizar fact demasiado pronto	Usarias (n+1)! en el termino n.
Usar math.factorial(n) en cada vuelta	Funciona, pero evita practicar el acumulador y recalcula trabajo que ya conocemos.
Confundir ex simbolico con ex numerico	Una expresion en x y su valor en xp son objetos conceptualmente distintos.
Pensar que el orden visual de SymPy es el orden del algoritmo	SymPy puede reordenar algebraicamente una suma sin cambiar su valor.